973. K Closest Points to Origin Medium

Acceptance: 68.6% | LeetCode

Given an array of `points` where `points[i] = [xi, yi]` represents a point on the **X-Y** plane and an integer `k`, return the `k` closest points to the origin `(0, 0)`.

The distance between two points on the **X-Y** plane is the Euclidean distance (i.e., `√(x1 - x2)2 + (y1 - y2)2`).

You may return the answer in **any order**. The answer is **guaranteed** to be **unique** (except for the order that it is in).

Example 1:

```
**Input:** points = [[1,3],[-2,2]], k = 1
**Output:** [[-2,2]]
**Explanation:**
The distance between (1, 3) and the origin is
sqrt(10).
The distance between (-2, 2) and the origin is
sqrt(8).
Since sqrt(8) < sqrt(10), (-2, 2) is closer to the
origin.
We only want the closest k = 1 points from the origin,
so the answer is just [[-2,2]].
```

Example 2:

```
**Input:** points = [[3,3],[5,-1],[-2,4]], k = 2
**Output:** [[3,3],[-2,4]]
**Explanation:** The answer [[-2,4],[3,3]] would also
be accepted.
```

Constraints:

- `1 <= k <= points.length <= 104`
- `-104 <= xi, yi <= 104`

Claude Opus Solution

K Closest Points to Origin

1. Intuition

The core insight is recognizing this as a **"top-K" selection problem**. We need the K closest points, but we **don't need them sorted** among themselves — we just need to *identify* which K points are closest.

Two key observations:

1. **Skip the square root**: Comparing `√(x² + y²)` is equivalent to comparing `x² + y²`. Since `√` is monotonically increasing, the *relative ordering* of distances is preserved. This avoids floating-point errors and is faster.
2. **For "K closest/largest/smallest" problems, a heap is the go-to pattern.** Specifically, to find the K *smallest* elements, we maintain a **max-heap of size K**. The heap holds the current K best candidates, and the root is the *worst* among them — so we can quickly decide whether a new point should replace it.

---

2. Approach

We use a **max-heap of size K** (Python only has min-heaps, so we negate distances to simulate a max-heap).

```
1. Initialize heap with the first K points (using -distance as the key).
2. heapify the list into a valid heap.
3. For each remaining point (index K to n-1):
   a. Compute its negated squared distance.
   b. If this distance > heap root (i.e., point is CLOSER than the
      farthest point currently in the heap):
        - Replace the root with this point (heappushpop).
4. The heap now contains the K closest points. Extract and return them.
```

Why "closer" means `distance > root`? We store *negated* distances. A point with smaller true distance → larger negated value. The min-heap root holds the smallest negated value = the largest true distance (the worst candidate). If a new point's negated distance is *greater* than the root, it's *closer*, so it deserves to be in the heap.

---

3. Python Solution

```python
import heapq
from typing import List


class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
```

```
        # ------------------------------------------------------------
        # Build a MAX-HEAP of size k using the first k points.
        # Python's heapq is a MIN-heap, so we negate the squared distance
        # to simulate a max-heap. This way the root (smallest negated value)
        # corresponds to the FARTHEST point among our k candidates.
        # ------------------------------------------------------------
        heap = [(-self._squared_distance(points[i]), i) for i in range(k)]
        heapq.heapify(heap)  # O(k) to build the heap in-place

        # ------------------------------------------------------------
        # Process the remaining points. For each one, check if it should
        # displace the current worst (farthest) candidate in the heap.
        # ------------------------------------------------------------
        for i in range(k, len(points)):
            neg_dist = -self._squared_distance(points[i])

            # heap[0][0] is the negated distance of the farthest candidate.
            # If our new point's negated distance is LARGER, it means the
            # new point is CLOSER to the origin -> swap it in.
            if neg_dist > heap[0][0]:
                # push the new point AND pop the old root in one efficient op
                heapq.heappushpop(heap, (neg_dist, i))

        # ------------------------------------------------------------
        # The heap now holds exactly the k closest points (in any order).
        # Return the actual point coordinates.
        # ------------------------------------------------------------
        return [points[i] for _, i in heap]

    def _squared_distance(self, point: List[int]) -> int:
        # Use squared distance to avoid sqrt: relative ordering is preserved.
        x, y = point
        return x * x + y * y
```

> **Note:** I store the *index* `i` instead of the point list itself in the heap. This avoids potential comparison issues if two points have identical distances (Python would try to compare the lists), and integers always compare cleanly.

---

4. Complexity Analysis

Metric	Complexity	Explanation
Time	`O(n log k)`	We `heapify` the first k in `O(k)`. Then for each of the remaining `n - k` points, we do at most one `heappushpop`, costing `O(log k)`. Total: `O(k + (n-k)·log k) = O(n log k)`.
Space	`O(k)`	The heap never exceeds size `k`.

This is **better than the naive `O(n log n)`** of fully sorting all points, especially when `k << n`.

---

5. Alternative Approaches

a) Sort everything — `O(n log n)` time, `O(n)` space

```
def kClosest(self, points, k):
    points.sort(key=lambda p: p[0]**2 + p[1]**2)
    return points[:k]
```

Simplest to write. Good when `k` is close to `n` or code clarity matters most.

b) QuickSelect (Partition-based) — `O(n)` average time, `O(n)` space

Use the partitioning logic of quicksort to position the k-th closest point so everything to its left is closer. We don't sort the full array — we only recurse into the side that contains the boundary. - **Average:** `O(n)` — beats the heap. - **Worst case:** `O(n²)` (bad pivots), mitigated by random pivot selection. - This is the optimal expected-time solution.

c) Binary Search on distance (Reference solution2) — `O(n)` time

Binary search on the *distance threshold* itself, repeatedly partitioning points into "closer" and "farther" buckets until exactly k closest are isolated. Each pass halves the search space, giving `O(n + n/2 + n/4 + ...) = O(n)`.

Summary Table

Approach	Time	Space	When to use
Sort	`O(n log n)`	`O(n)`	Quick to write, k ≈ n
Heap (this)	`O(n log k)`	`O(k)`	k ≪ n, streaming data
QuickSelect	`O(n)` avg	`O(n)`	Best avg performance
Binary Search	`O(n)`	`O(n)`	Clever, but complex

The heap approach is the recommended default — it has predictable performance, is memory-efficient when `k` is small, and naturally extends to streaming scenarios where points arrive one at a time.

Codex Solution

1. Intuition

We need the `k` points with the smallest distance to `(0, 0)`.

For a point `[x, y]`, its Euclidean distance from the origin is:

```
sqrt(x^2 + y^2)
```

However, we do **not** need the actual distance. Since square root is monotonic, comparing:

```
sqrt(a) < sqrt(b)
```

is equivalent to comparing:

```
a < b
```

So we can compare points using their **squared distance**:

```
x^2 + y^2
```

The key pattern is:

> Maintain the best `k` candidates seen so far using a heap.

Because Python only has a min-heap, we simulate a **max-heap** by storing negative distances. This lets us keep the farthest point among our current `k` closest points at the top of the heap, so we can easily remove it when we find a closer point.

2. Approach

We use a heap of size at most `k`.

Pseudocode

```
function kClosest(points, k):
    create empty heap

    for each point [x, y] in points:
        dist = x^2 + y^2

        if heap has fewer than k points:
            push (-dist, x, y) into heap

        else:
            farthest_distance_in_heap = -heap[0].distance

            if current point is closer than farthest point in heap:
                remove farthest point from heap
                add current point to heap

    return all points stored in heap
```

Why negative distance?

```
heapq is a min-heap
smallest value is at heap[0]
```

```
If we store -distance:
    larger actual distance becomes smaller negative value

Example:
    distance = 10 -> -10
    distance = 5  -> -5

heap[0] will be -10, representing the farthest point
```

3. Python Solution

```python
from typing import List
import heapq


class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        # Heap will store tuples of the form:
        # (-squared_distance, x, y)
        #
        # We use negative distance because Python's heapq is a min-heap.
        # By storing negative distances, the "smallest" value in the heap
        # actually represents the largest actual distance.
        #
        # Therefore, heap[0] is always the farthest point among the current
        # k closest points we have kept.
        heap = []

        for x, y in points:
            # Squared distance from origin:
            # We avoid sqrt because sqrt is unnecessary for comparison.
            dist = x * x + y * y

            # If we still have fewer than k points, add this point.
            if len(heap) < k:
                heapq.heappush(heap, (-dist, x, y))

            else:
                # heap[0] contains the farthest point among the current k points.
                # Its stored distance is negative.
                farthest_dist = -heap[0][0]

                # If current point is closer than the farthest point in heap,
                # remove the farthest point and insert the current one.
                if dist < farthest_dist:
                    heapq.heapreplace(heap, (-dist, x, y))

        # Extract the points from the heap.
        # The answer can be returned in any order.
        return [[x, y] for _, x, y in heap]
```

4. Complexity Analysis

Let:

```
n = number of points
```

Time Complexity

```
O(n log k)
```

Explanation:

- We process every point once.
- The heap size is at most `k`.
- Each heap operation costs `O(log k)`.
- In the worst case, we may perform a heap operation for each point.

So total time is:

```
O(n log k)
```

Space Complexity

```
O(k)
```

Explanation:

- The heap stores at most `k` points.
- We do not store all `n` points in an extra data structure.

---

5. Alternative Approaches

1. Sorting

Sort all points by squared distance and return the first `k`.

```python
class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        points.sort(key=lambda p: p[0] * p[0] + p[1] * p[1])
        return points[:k]
```

Complexity:

```
Time:  O(n log n)
Space: O(1) or O(n), depending on sorting implementation details
```

This is the simplest solution and works well for the given constraints.

---

2. Min-Heap of All Points

Push every point into a min-heap by distance, then pop `k` points.

Complexity:

```
Time:  O(n + k log n)
Space: O(n)
```

This is useful if you naturally want to repeatedly retrieve the next closest point.

---

3. Quickselect

Use the partition idea from quicksort to place the `k` closest points in the first `k` positions.

Complexity:

```
Average Time: O(n)
Worst Time:   O(n^2)
Space:        O(1) or O(log n), depending on implementation
```

This is usually the most optimal average-time approach, but it is more complex than heap or sorting.